

DOCUMENT AI SYSTEM

Technical Architecture and Design

The Document AI system is an AI-powered document analysis and question answering application.

Users upload PDF documents through a Streamlit interface. The backend processes documents, creates searchable representations, and allows users to ask natural-language questions about uploaded documents.

The application consists of a Streamlit frontend, FastAPI backend, PDF processing services, an embedding service, a vector database, and a large language model.

RAG PIPELINE

The system uses Retrieval-Augmented Generation, commonly called RAG, to answer questions about documents.

During ingestion, the PDF is converted into text, divided into smaller chunks, embeddings are created, and embeddings are stored in a vector database.

During question answering, the user question is converted into an embedding. A similarity search retrieves the most relevant document chunks, which are then provided as context to the language model.

EMBEDDINGS

Embeddings are numerical representations of text that capture semantic meaning.

Similar pieces of text tend to have similar vector representations.

The system uses Sentence Transformers and the all-MiniLM-L6-v2 embedding model.

Every document chunk and every user question is converted into an embedding before similarity search.

VECTOR DATABASE

The vector database stores document chunks together with their embeddings and metadata.

Metadata can include the filename, page number, chunk identifier, and username.

When a user asks a question, the system searches the vector database for chunks semantically similar to the question.

PDF PROCESSING

PDF processing begins when a user uploads a PDF document.

The application validates the file type and file size before processing it.

Text is extracted from individual pages. The extracted text is divided into overlapping chunks.

CHUNKING

Chunking divides a long document into smaller pieces that can be embedded and retrieved efficiently.

The system uses chunks of approximately 1000 characters with an overlap of approximately 200 characters.

Overlap helps preserve context when an important sentence occurs near a chunk boundary.

QUESTION ANSWERING

A user selects a document and enters a natural-language question.

The question is converted into an embedding using the same embedding model used for document chunks.

The vector database performs similarity retrieval and returns relevant chunks.

A relevance threshold helps prevent unrelated chunks from being passed to the language model.

LANGUAGE MODEL

The language model receives the user question together with retrieved document context.

The model is instructed to answer using the supplied document context rather than inventing information.

The application uses Groq as the language-model provider.

The configured model is openai/gpt-oss-20b.

SECURITY AND AUTHENTICATION

The application uses JWT-based authentication.

Users register with a username and password. Passwords are stored as secure password hashes rather than plain text.

After successful login, the backend issues an access token.

Protected endpoints require the token, and document metadata is filtered by authenticated username.

SYSTEM SUMMARY

The complete pipeline is: Upload PDF -> Extract Text -> Chunk Text -> Create Embeddings -> Store in ChromaDB -> Retrieve Relevant Chunks -> Send Context to LLM -> Generate Answer.

The architecture separates the frontend, API routes, processing services, vector storage, authentication, and language-model generation.

This document is intended for testing PDF extraction, chunking, embeddings, vector retrieval, RAG generation, source metadata, and authentication-aware document access.